

Lab Documentation*Team A8**01.07.2026*

1 Team members.....	1
2 Concept description.....	2
2.1 Instruction format.....	2
2.2 Instruction set.....	2
2.3 State machine.....	3
3. Tools and Technologies.....	3
4. Project/Team management and Design Conventions.....	4
4.1. VHDL and FPGA Design.....	4
5. Architecture Overview.....	4
6. VHDL Implementation.....	5
6.1 CPU_Package.vhd.....	5
6.2 Program_Counter.vhd.....	5
6.3 Instruction_Register.vhd.....	6
6.4 Accumulator.vhd.....	7
6.5 ALU.vhd.....	8
6.6 Memory_Unit.vhd.....	9
6.7 Control_Unit.vhd.....	10
6.8 CPU_Top.vhd.....	11
6.9 Board Integration Modules.....	13
6.9.1 Clock_Divider.vhd.....	13
6.9.2 Button_Debouncer.vhd.....	14
6.9.3 Hex_To_SevenSegment.vhd.....	15
6.9.4 SevenSegment_Driver.vhd.....	15
6.9.5 Nexys_A7_Top.vhd.....	16
7. Testbench Verification.....	18
8. FPGA Implementation.....	19
9. PCB Design.....	19
10. Results.....	20
Sources/References.....	22

1 Team members

Talha Khan
 Muhammad Ali
 Moezz Mufti
 Muhammad Hamas

2 Concept description

The aim of this project is to design, simulate and implement a small 8-bit processor in VHDL, and to run it on an FPGA development board (Digilent Nexys A7-100T). The design follows a Harvard architecture: program instructions and data are held in two separate memories, so an instruction can be fetched at the same time as data is accessed.

Instructions are pre-loaded into a program memory (ROM). The CPU fetches each instruction, decodes it, executes the operation in an arithmetic-logic unit, and shows the result on the board's seven-segment display. A small set of demonstration programs is built into the ROM and selected at runtime with two switches.

The processor is a single-accumulator machine: one working register (the accumulator) holds the current value, and every operation works on the accumulator together with an immediate value or a value read from data memory.

2.1 Instruction format

Every instruction is one byte (8 bits). The upper four bits are the opcode (which operation), and the lower four bits are the operand — either a small immediate value or a 4-bit memory address.

bits 7..4 = opcode bits 3..0 = operand / address

Example: the instruction 0001 0011 means LOAD_IMMEDIATE with operand 3, i.e. load the value 3 into the accumulator.

2.2 Instruction set

The opcode is a 4-bit field, giving 16 possible operations. All sixteen are defined once in the package file (see Section 5.1) and used by name throughout the design.

Opcode	Mnemonic	Operation
0x0	NOP	No operation
0x1	LOAD_IMMEDIATE	ACC $\leftarrow$ operand
0x2	LOAD_MEMORY	ACC $\leftarrow$ RAM[address]
0x3	STORE_MEMORY	RAM[address] $\leftarrow$ ACC
0x4	ADD_MEMORY	ACC $\leftarrow$ ACC + RAM[address]
0x5	SUB_MEMORY	ACC $\leftarrow$ ACC – RAM[address]
0x6	AND_MEMORY	ACC $\leftarrow$ ACC AND RAM[address]
0x7	OR_MEMORY	ACC $\leftarrow$ ACC OR RAM[address]
0x8	XOR_MEMORY	ACC $\leftarrow$ ACC XOR RAM[address]
0x9	NOT_ACC	ACC $\leftarrow$ NOT ACC
0xA	JUMP	PC $\leftarrow$ address

Opcode	Mnemonic	Operation
0xB	JUMP_IF_ZERO	if Zero flag: PC $\leftarrow$ address
0xC	JUMP_IF_CARRY	if Carry flag: PC $\leftarrow$ address
0xD	OUT	Output register $\leftarrow$ ACC
0xE	CLEAR	ACC $\leftarrow$ 0
0xF	HALT	Stop execution

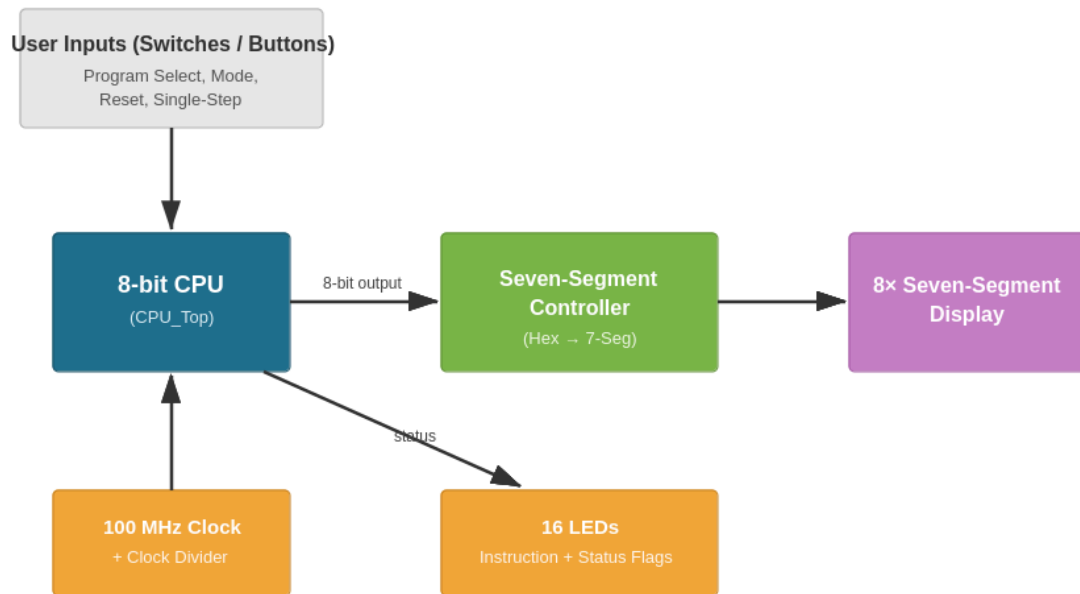


Figure 1: Top-level block diagram .

2.3 State machine

The control unit steps through a fetch / decode / execute cycle. From reset it begins in FETCH, advances through DECODE and EXECUTE, then returns to FETCH for the next instruction. A HALT instruction moves the machine into a HALT state, where it stays until reset.

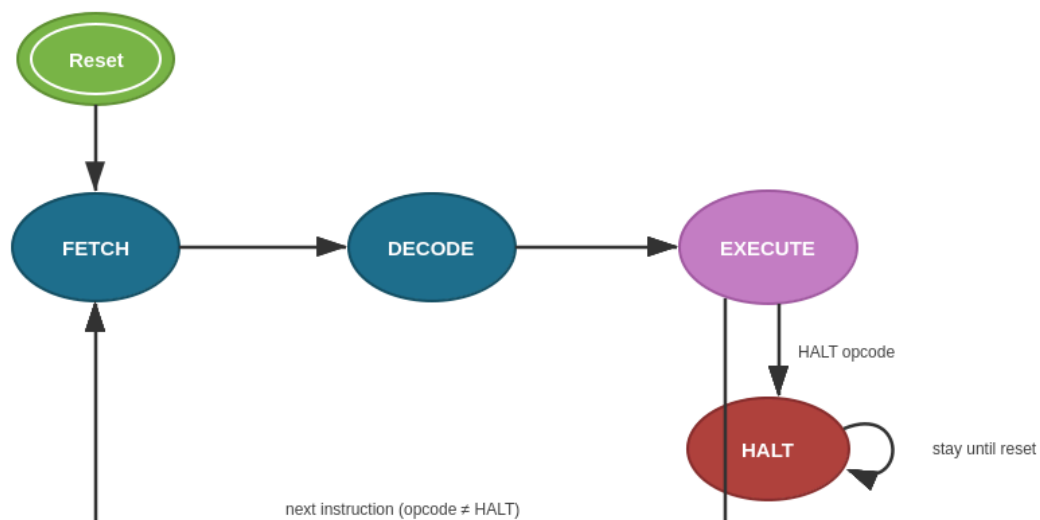


Figure 2: State diagram: Reset $\rightarrow$ FETCH $\rightarrow$ DECODE $\rightarrow$ EXECUTE .

3. Tools and Technologies

Tool / technology	Purpose
VHDL	Hardware description language used for all modules
Xilinx Vivado	Synthesis, implementation, simulation and bitstream generation
Nexys A7-100T (Artix-7)	FPGA development board used to run and testing the design
GitHub	Version control and central storage of project files

The IEEE standard libraries are used throughout: `std_logic_1164` for the logic types (`std_logic`, `std_logic_vector`) and `numeric_std` for arithmetic on unsigned values.

4. Project/Team management and Design Conventions

To keep the modules consistent and easy to read, the same conventions are used in every file:

- Asynchronous reset: every register is cleared immediately when Reset is asserted, independent of the clock.
- Synchronous clock enable: a single Clock_Enable gates every register, so the whole CPU can be advanced one cycle at a time (used for single-stepping on the board).
- Rising-edge clocking via `rising_edge(Clock)`.
- The control unit is built as a three-process finite-state machine (a separate state register, next-state logic, and output logic).
- Each file begins with a header comment describing its purpose, and important lines are commented.

4.1. VHDL and FPGA Design

The digital-design part of the project followed the following approach: requirements were fixed first, followed by a functional block diagram, then VHDL implementation of each module. Verification mirrored the design hierarchy: every module was checked with its own self-checking testbench, the assembled CPU was checked with a top-level testbench running the demonstration programs, and the final design was validated on the FPGA board.

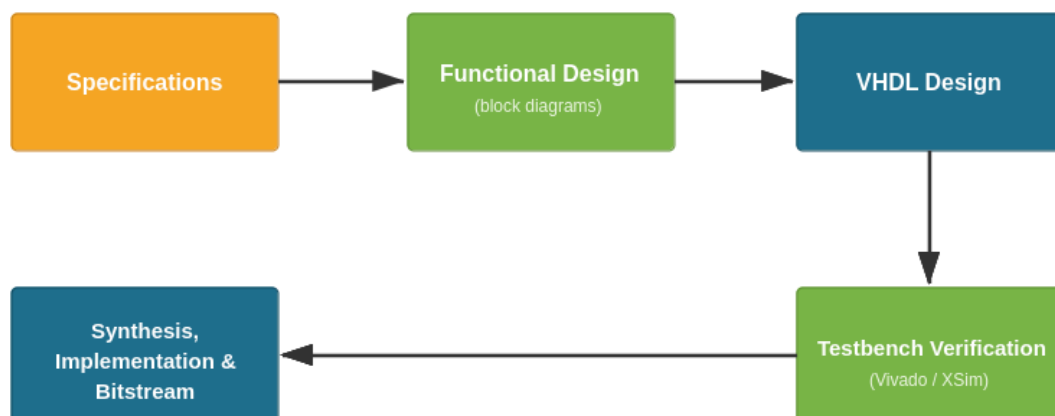


Figure 3: Digital-design workflow diagram.

5. Architecture Overview

The CPU is built from six core modules plus a shared definitions package. A structural top level (CPU_Top) wires them together and exposes a set of debug signals for observation. The datapath flows as follows: the program counter addresses the program ROM; the fetched instruction is latched in the instruction register and split into opcode and operand; the control unit decodes the opcode and drives the datapath; the ALU computes a result from the accumulator and a second operand; and the accumulator stores the result.

6. VHDL Implementation

This section describes every source file: its role, how it works, and a placeholder for its Vivado schematic. Unless stated otherwise, each schematic is produced the same way:

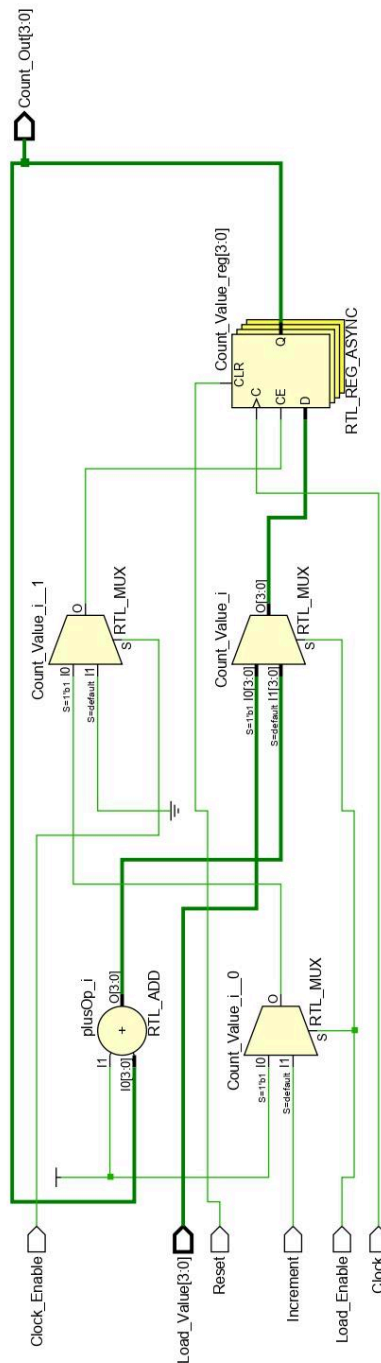
6.1 CPU_Package.vhd

This is a shared design unit (a VHDL package). It does not describe hardware on its own; instead it holds the definitions that every other module needs in one place: the sixteen opcode constants (OP_NOP ... OP_HALT) and the control-unit state type. Centralising these means the opcodes are written once and referred to by name, which avoids mismatched values between modules.

6.2 Program_Counter.vhd

The program counter (PC) holds the address of the next instruction to fetch. It is a 4-bit counter that can either increment by one (normal sequential execution) or load a new value (a jump). Load has priority over increment, so a jump always overrides the normal +1.

- Asynchronous reset sets the count to 0 (restart from the first instruction).
- On a rising clock edge, if Clock_Enable is high: load the jump address when Load_Enable is set, otherwise add one when Increment is set.
- The count is kept as an unsigned value internally and converted back to std_logic_vector on the output.



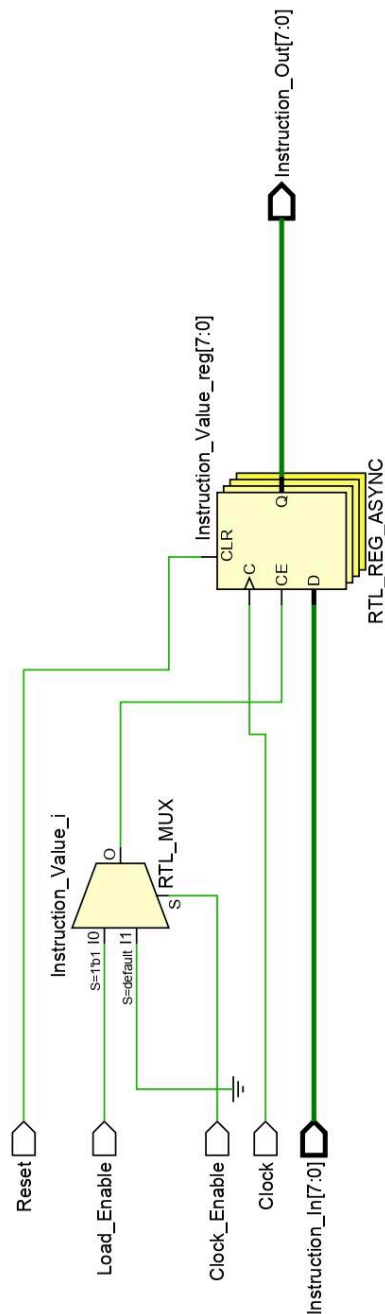
Program_Counter elaborated-design schematic.

Figure 4: Program_Counter elaborated-design schematic.

6.3 Instruction_Register.vhd

The instruction register (IR) latches the instruction word that was fetched from the program ROM, so it stays stable while it is decoded and executed. It is an 8-bit register that captures its input only when Load_Enable is high (asserted during the fetch step); otherwise it holds its value.

- Asynchronous reset clears it to 0.
- Captures Instruction_In on a rising edge when both Clock_Enable and Load_Enable are high.



Instruction_Register elaborated-design schematic.

Figure 5: *Instruction_Register elaborated-design schematic.*

6.4 Accumulator.vhd

The accumulator (ACC) is the CPU's main working register. It holds the value that operations act on, and it is where the ALU result is written back. It is an 8-bit register with the same load-on-enable behaviour as the instruction register: it loads the ALU result when ACC_Load is high, and holds otherwise.

- Asynchronous reset clears it to 0.
- Loads Data_In (the ALU result) on a rising edge when Clock_Enable and Load_Enable are high.

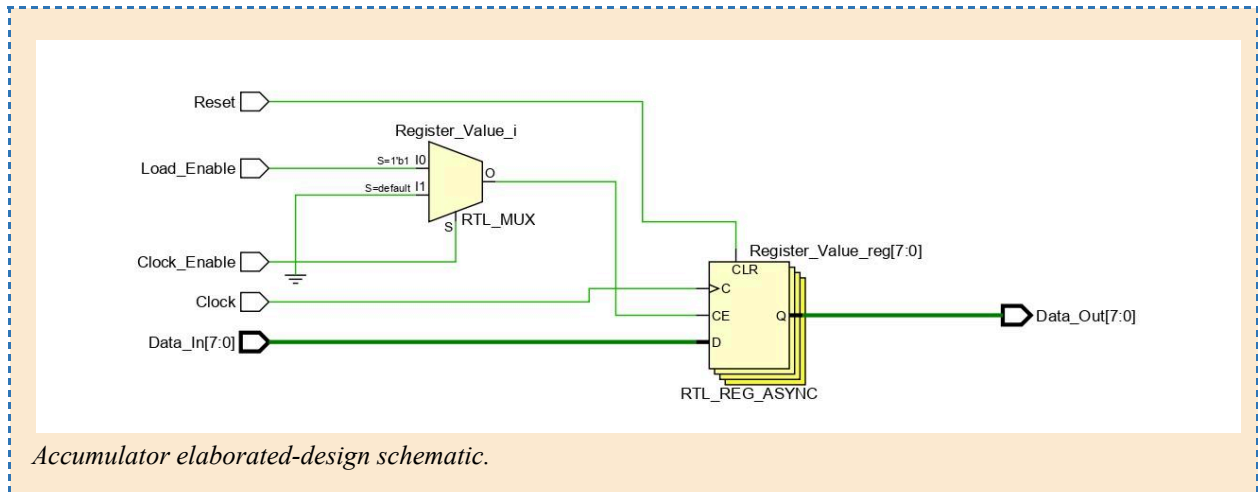
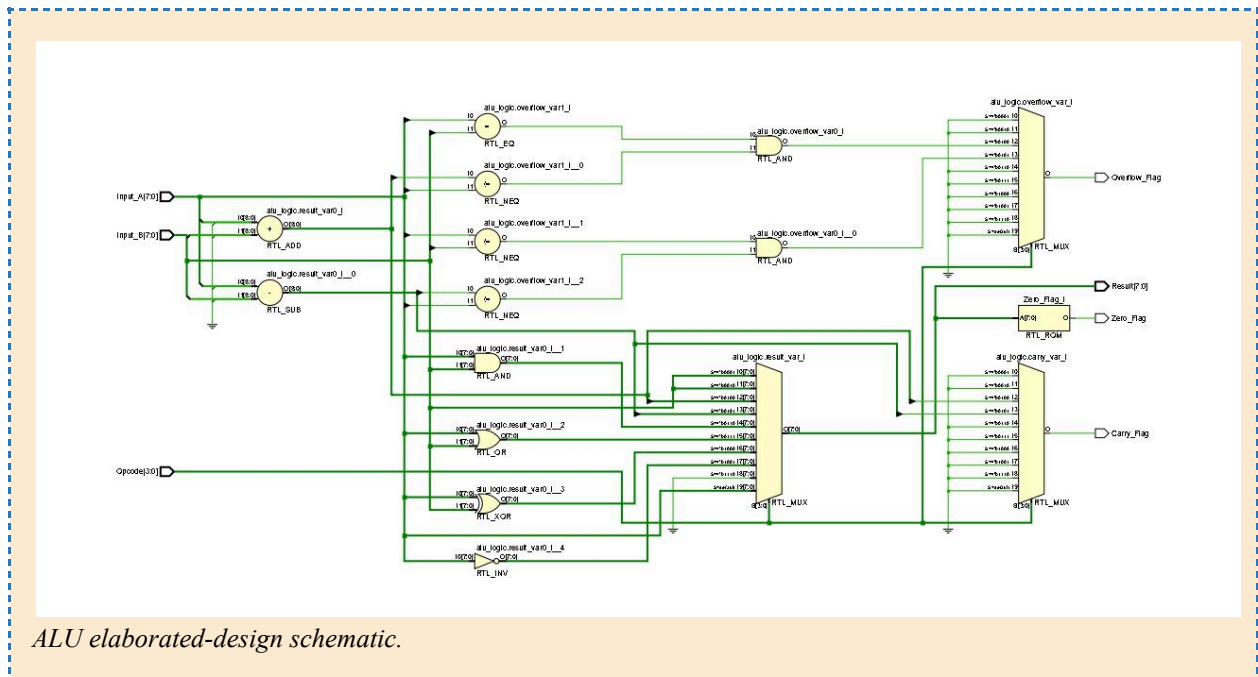


Figure 6: Accumulator elaborated-design schematic.

6.5 ALU.vhd

The arithmetic-logic unit (ALU) is purely combinational. It takes the accumulator (Input_A), a second operand (Input_B) and the opcode, and produces an 8-bit result together with three status flags. The operation is chosen by a case statement on the opcode.

- Arithmetic: addition and subtraction. A 9-bit intermediate value is used so the ninth bit captures the carry / borrow out of the 8-bit operation.
- Logic: AND, OR, XOR, NOT, and CLEAR.
- Load operations pass the second operand straight through to the result.
- Flags: Carry (carry/borrow out), Zero (result is all zeros), and Overflow (signed overflow on add/subtract).



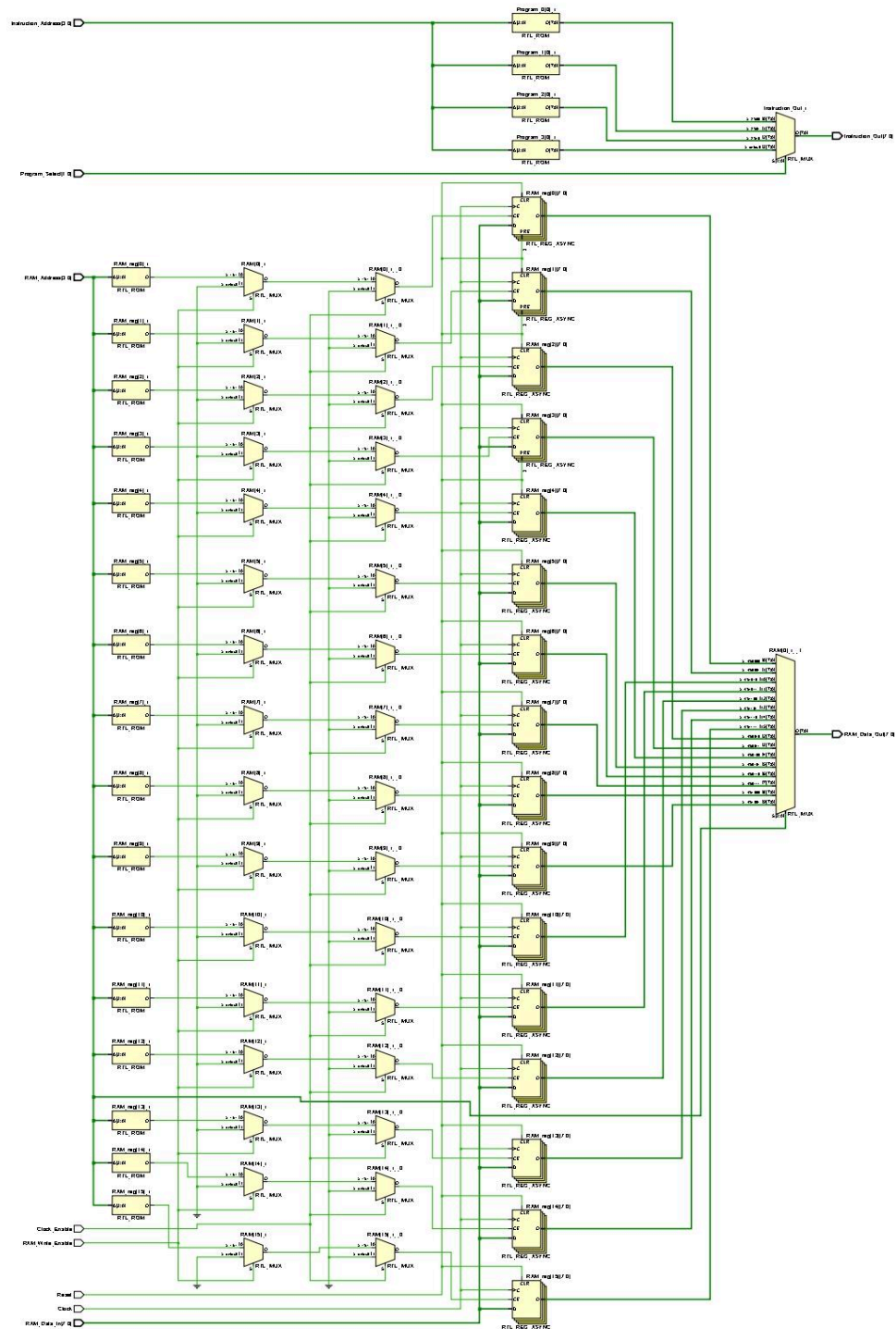
ALU elaborated-design schematic.

Figure 7: ALU elaborated-design schematic.

6.6 Memory_Unit.vhd

The memory unit contains both memories of the Harvard architecture:

- Program ROM — read-only. It holds four demonstration programs; the 2-bit Program_Select input chooses which one. It is addressed by the program counter and read combinationally.
- Data RAM — 16 bytes of read/write memory used by the load, store and arithmetic instructions. Reads are combinational (asynchronous); writes happen on a rising clock edge when enabled. The RAM start-up contents are kept in a single constant, and an asynchronous reset restores them.



Memory_Unit elaborated-design schematic (program ROM + data RAM).

Figure 8: Memory_Unit elaborated-design schematic (program ROM + data RAM).

6.7 Control_Unit.vhd

The control unit is the “brain” of the CPU. It implements the finite-state machine that sequences fetch / decode / execute, and in each state it raises the control signals that steer the datapath (latch the

instruction, increment or load the PC, load the accumulator, write to RAM, load the output register, or halt).

It is written using the three-process method:

- State register — a clocked process that stores the current state (asynchronous reset, synchronous enable).
- Next-state logic — a combinational process that decides the next state from the current state and the opcode.
- Output logic — a combinational process that sets the control outputs for the current state. A HALT opcode in EXECUTE causes a move to the HALT state, where the halt signal is raised.

The current state is also exposed on State_Debug so it can be observed on the board.

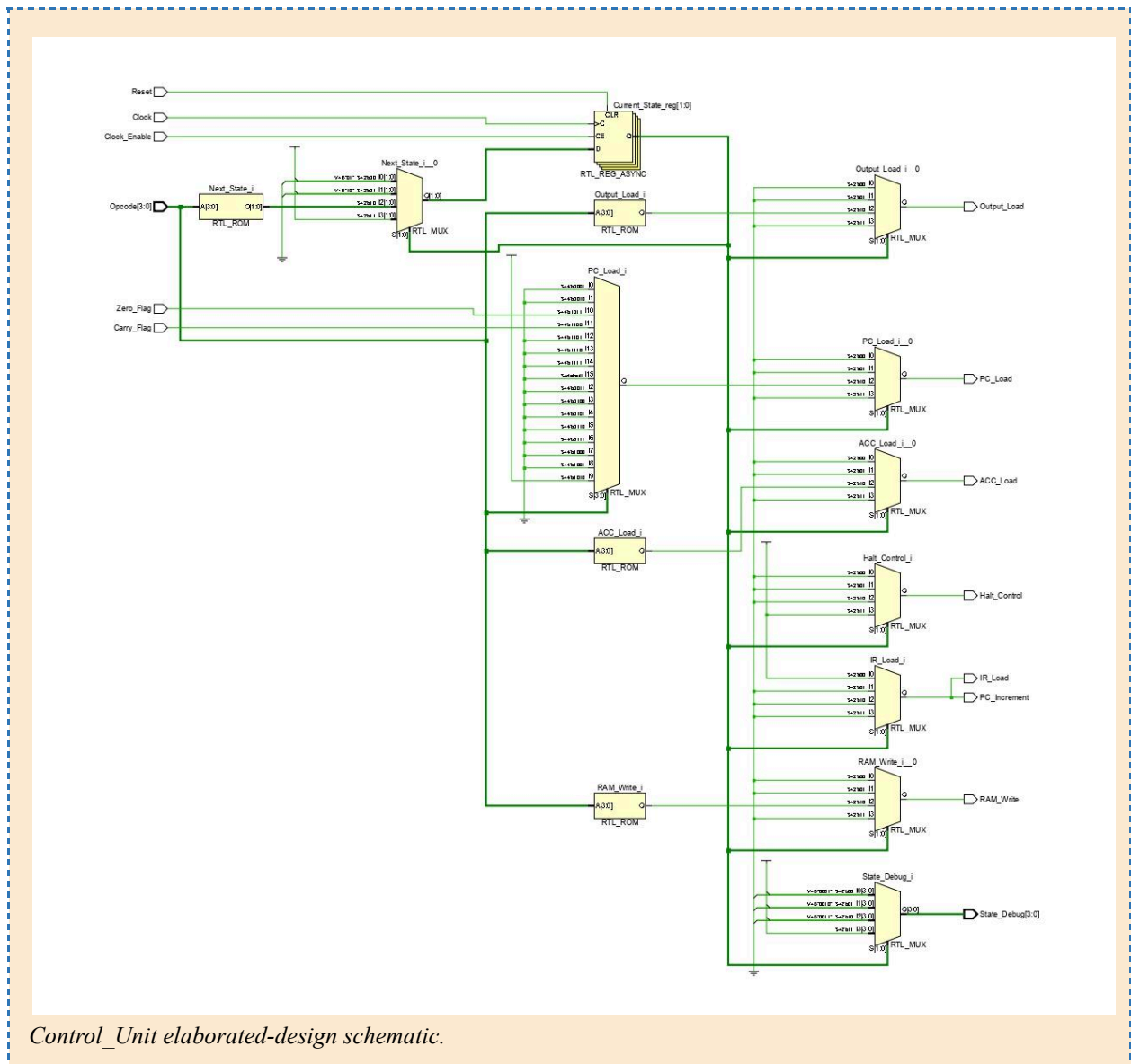


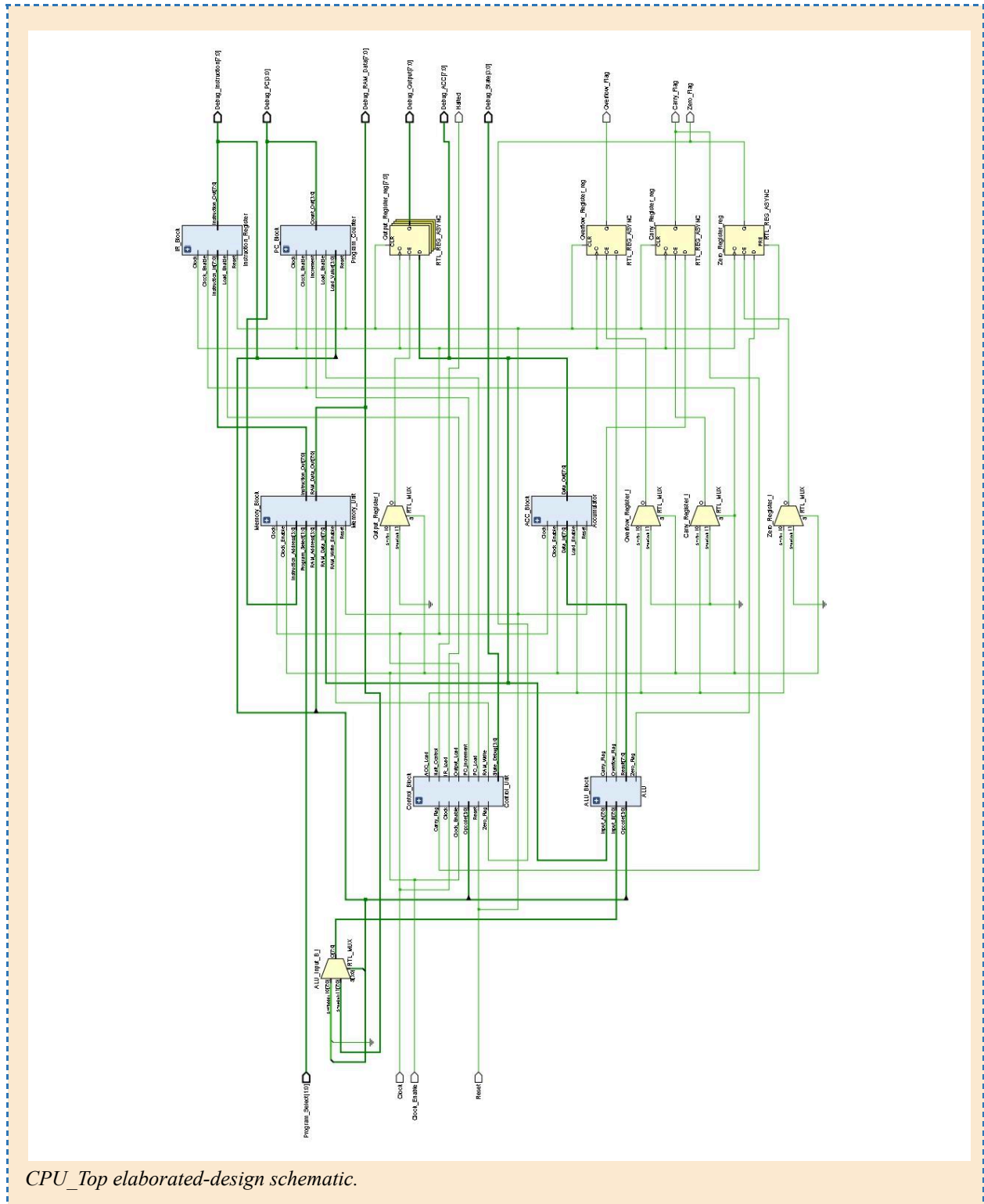
Figure 9: Control_Unit elaborated-design schematic.

6.8 CPU_Top.vhd

CPU_Top is the structural top level of the processor core. It instantiates the six modules above and connects them into the complete datapath and control path. It also contains a small status/output

register that latches the flags when the accumulator updates (so conditional jumps see the correct flags) and the output value when an OUT instruction runs.

- Splits the latched instruction into opcode (bits 7..4) and operand (bits 3..0).
- Selects the ALU's second operand: an immediate value for LOAD_IMMEDIATE, otherwise the value read from data RAM.
- Exposes debug outputs (PC, instruction, state, accumulator, output, RAM data) and the status flags / halted signal for observation.



CPU_Top elaborated-design schematic.

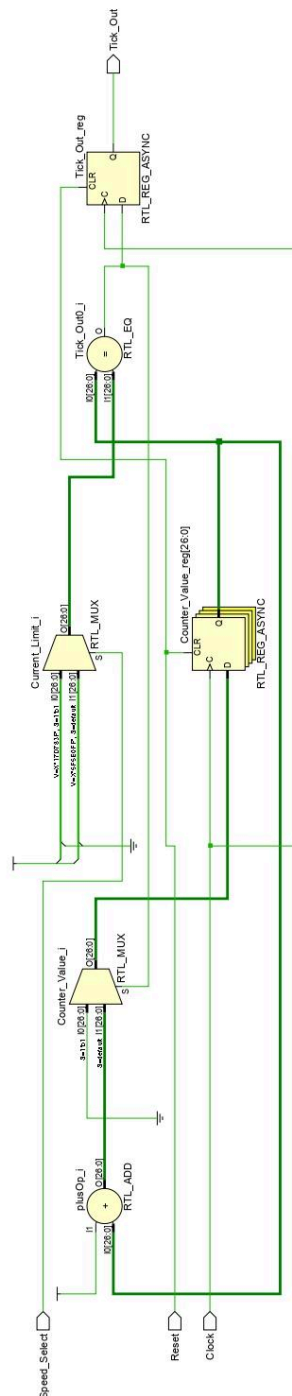
Figure 10: CPU_Top elaborated-design schematic.

6.9 Board Integration Modules

These modules wrap the CPU core so it can run on the Nexys A7-100T, driving the seven-segment display and LEDs and providing a clean clock-enable for stepping.

6.9.1 Clock_Divider.vhd

Produces a single-cycle pulse every N clock cycles. On the board it generates a slow “free-run” enable (a few cycles per second) so the fetch/decode/execute steps are slow enough to watch. The divisor is a generic, so it can be set small in simulation.

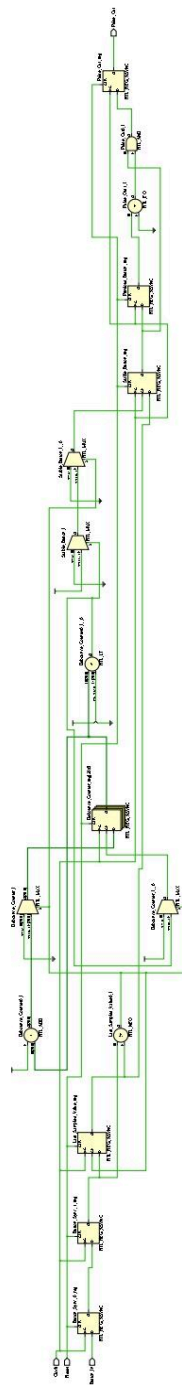


Clock_Divider elaborated-design schematic.

Figure 11: Clock_Divider elaborated-design schematic.

6.9.2 Button_Debouncer.vhd

Cleans up a mechanical pushbutton and turns one physical press into exactly one single-cycle pulse, used to single-step the CPU. It synchronises the button into the clock domain with two flip-flops, waits until the level has been stable for a set time (debounce), and then emits one pulse on the rising edge.



Button_Debouncer elaborated-design schematic.

Figure 12: Button_Debouncer elaborated-design schematic.

6.9.3 Hex_To_SevenSegment.vhd

A combinational decoder that converts a 4-bit value (0–F) into the seven-segment pattern for that hex digit. The Nexys A7 display is common-anode, so the segment outputs are active-low (a 0 lights a segment).

6.9.4 SevenSegment_Driver.vhd

Drives the 8-digit display by time-multiplexing: it lights one digit at a time but cycles through all eight fast enough (about 95 Hz over the whole display) that the eye sees them all at once. A refresh counter selects the active digit; the matching nibble is decoded by a single shared hex decoder, and a one-hot active-low signal enables the correct anode. Digits can be individually blanked.

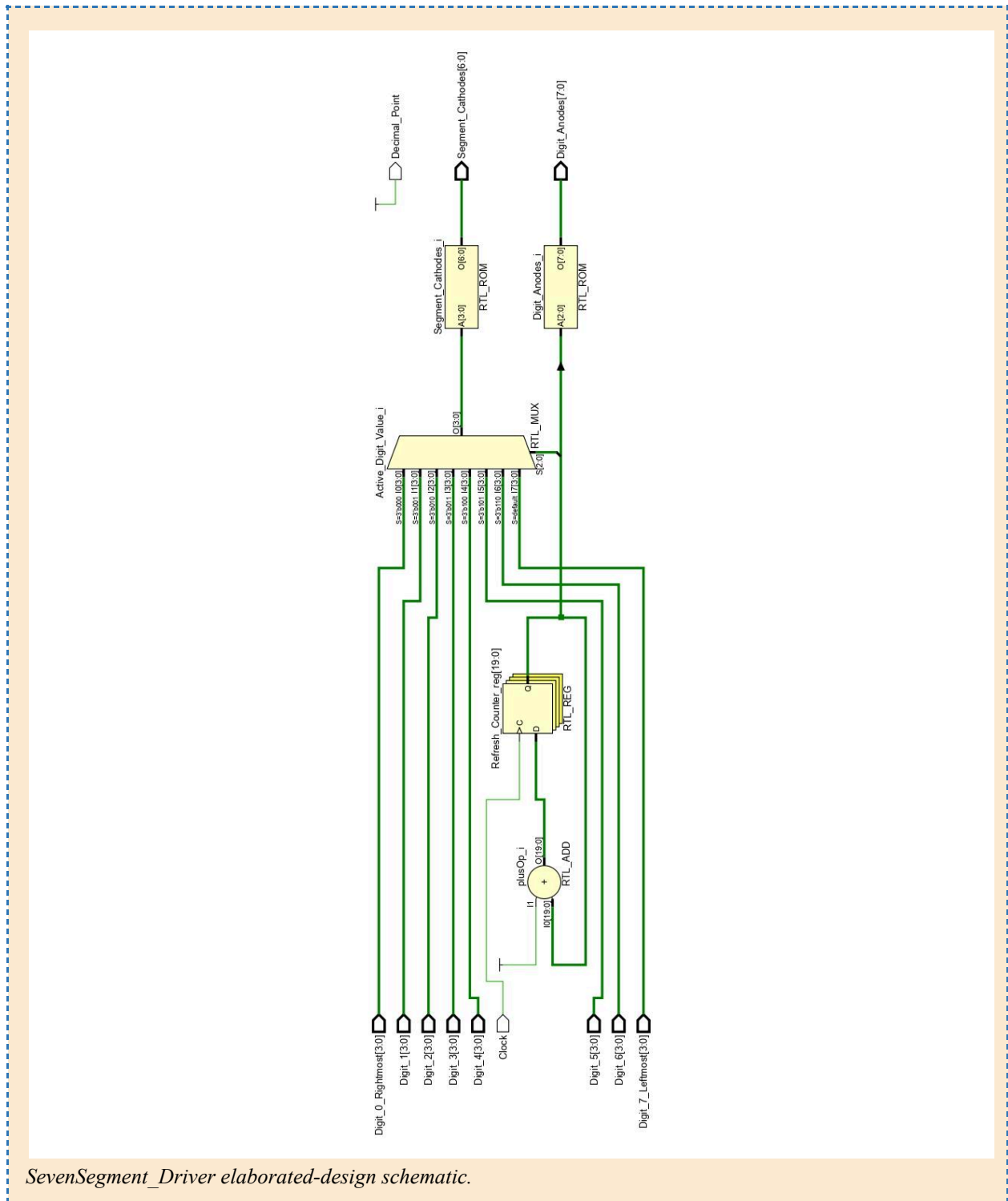


Figure 13: SevenSegment Driver elaborated-design schematic.

6.9.5 Nexys_A7_Top.vhd

The board top level. It instantiates the CPU core and the display/clock/button modules and maps everything to the physical I/O. A mode switch selects how the CPU advances: free-run (slow automatic tick) or single-step (one CPU cycle per button press). The display shows, in hex, the

program counter, the FSM state, and the output value; the LEDs show the current instruction and the status flags.

Board control	Function
CPU_RESETN (button)	Reset (active low)
SW(1..0)	Program select (which demo program)
SW(15)	Mode: 0 = single-step, 1 = free-run
BTNC (button)	Single-step (one press = one CPU step)
Seven-segment	PC / state / output value (hex)
LED(7..0)	Current instruction
LED(8..11)	Zero, Carry, Overflow, Halted

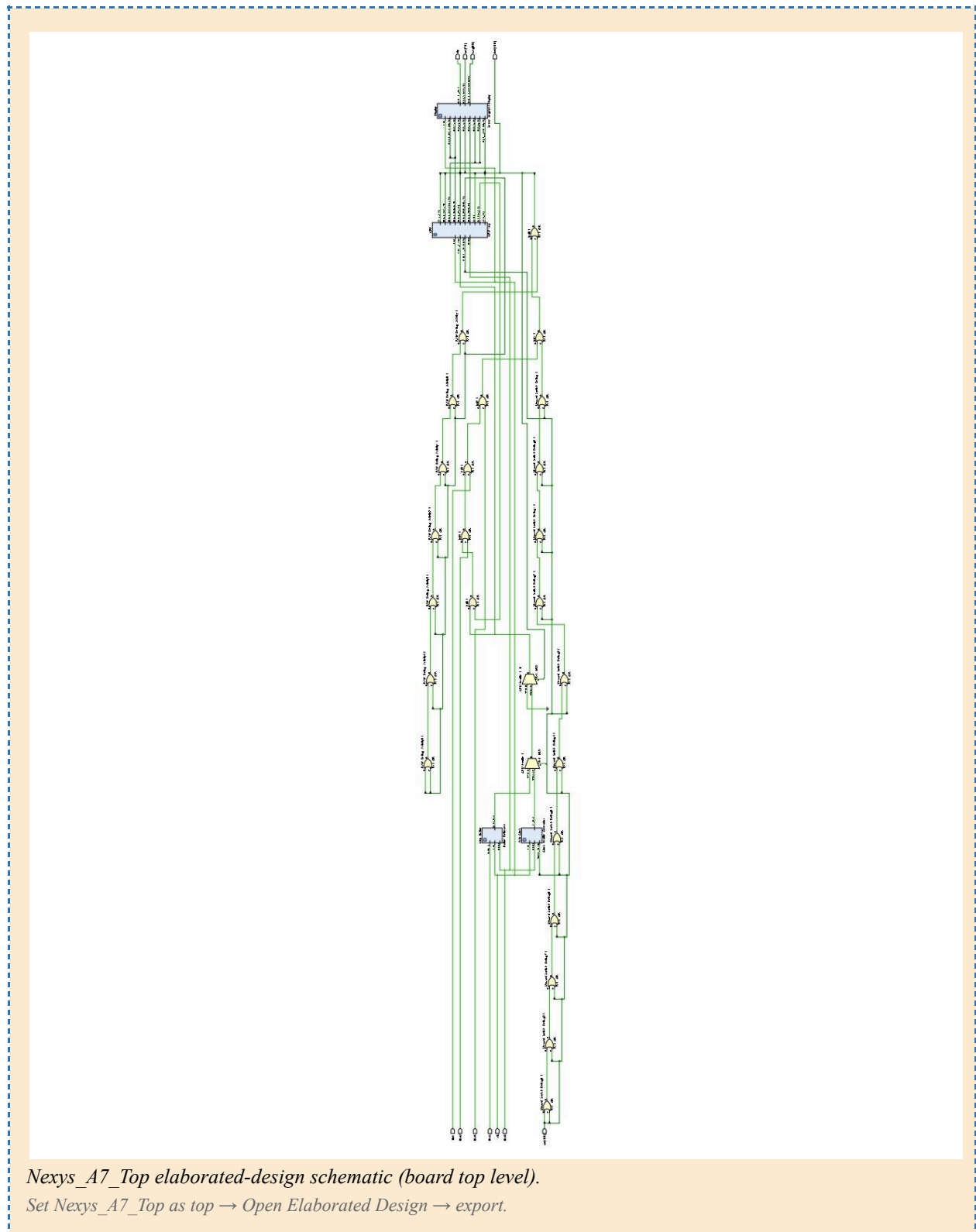


Figure 14: Nexys_A7_Top elaborated-design schematic (board top level).

7. Testbench Verification

Every module has its own self-checking testbench, and a top-level testbench runs all four demonstration programs end-to-end. Each test applies known inputs and uses assert / report statements to compare the result against the expected value, printing a PASS line (or raising an error). The console output is therefore the pass/fail record; waveforms are kept as supporting evidence.

Testbench	Checks	What it verifies
Program_Counter_tb	7	reset, increment, enable gating, jump, jump priority
Instruction_Register_tb	5	reset, load, hold, enable gating
Accumulator_tb	5	reset, load, hold, enable gating
ALU_tb	11	every operation + carry / zero / overflow flags
Memory_Unit_tb	8	ROM decode, RAM init / write / gating / reset
Control_Unit_tb	13	FSM order, opcode decode, halt entry
CPU_Top_tb	4	all four demo programs, end to end

The four demonstration programs and their expected results are:

Program_Select	Demonstrates	Final output
00	arithmetic + load/store + output	0x04
01	8-bit datapath (0F + 01 = 10)	0x10
10	logic operations	0x0F
11	conditional jump on the zero flag	0x07

8. FPGA Implementation

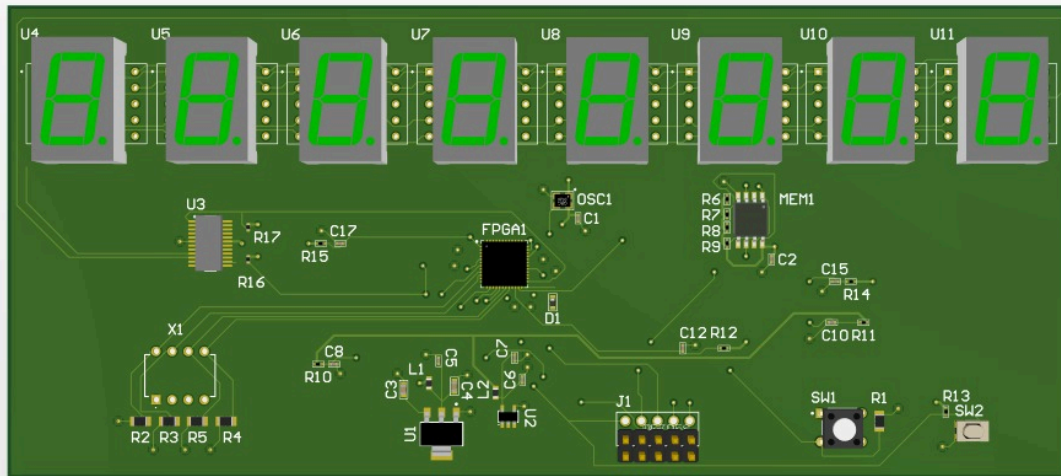
After simulation, the design was synthesised and implemented in Vivado for the Nexys A7-100T, and a bitstream was generated and loaded onto the board. The board top level (Nexys_A7_Top) connects the CPU to the switches, buttons, LEDs and seven-segment display through the constraints file.

On the board, a program is selected with the switches and the CPU is advanced either automatically (free-run) or one step at a time (single-step button). The seven-segment display shows the program counter, the state and the output value, and the result of each demonstration program matches the simulated value.

9. PCB Design

[This section will document the custom PCB. To be completed.]

Planned contents: design objectives, component selection, schematic (hierarchical sheets), PCB layout and placement strategy, ERC/DRC validation, and generation of manufacturing (Gerber) files.



PCB schematic and layout.

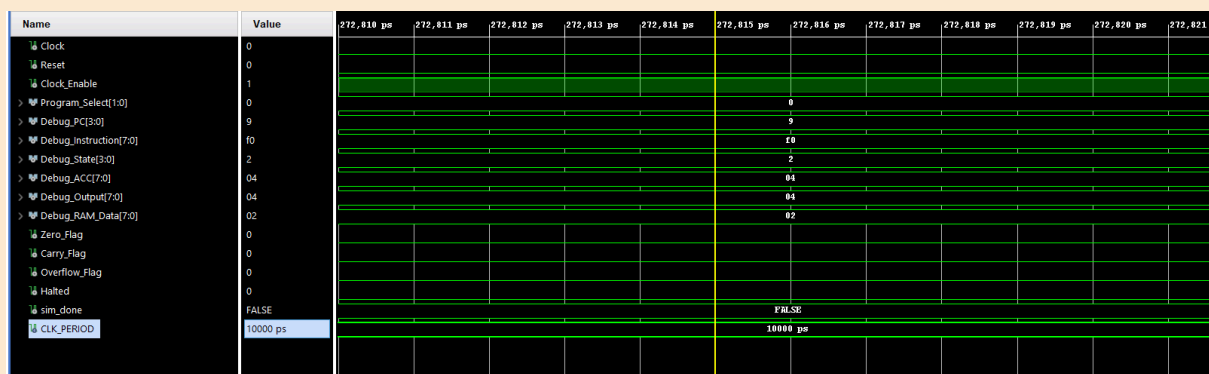
To be added once the PCB is designed.

Figure 15: PCB schematic and layout.

10. Results

The processor was verified first in simulation and then on the FPGA board. At the module level, every component has its own self-checking testbench, and a top-level testbench exercises the complete CPU. Across the seven testbenches, all 53 assertion checks pass with no failures, confirming that each block behaves as specified and that the assembled processor executes instructions correctly.

The four demonstration programs were run through the top-level testbench, and each produces its expected result before halting: Program 00 outputs **0x04** (arithmetic with load/store), Program 01 outputs **0x10** (the 8-bit addition 0F + 01, a result a 4-bit datapath could not produce), Program 10 outputs **0x0F** (a chain of logic operations), and Program 11 outputs **0x07** (a conditional jump taken on the zero flag). These results are shown in Figures 16–19.



Top-level testbench result (console PASS lines and/or waveform) for Program 00.

Figure 16: Top-level testbench result (console PASS lines and/or waveform).

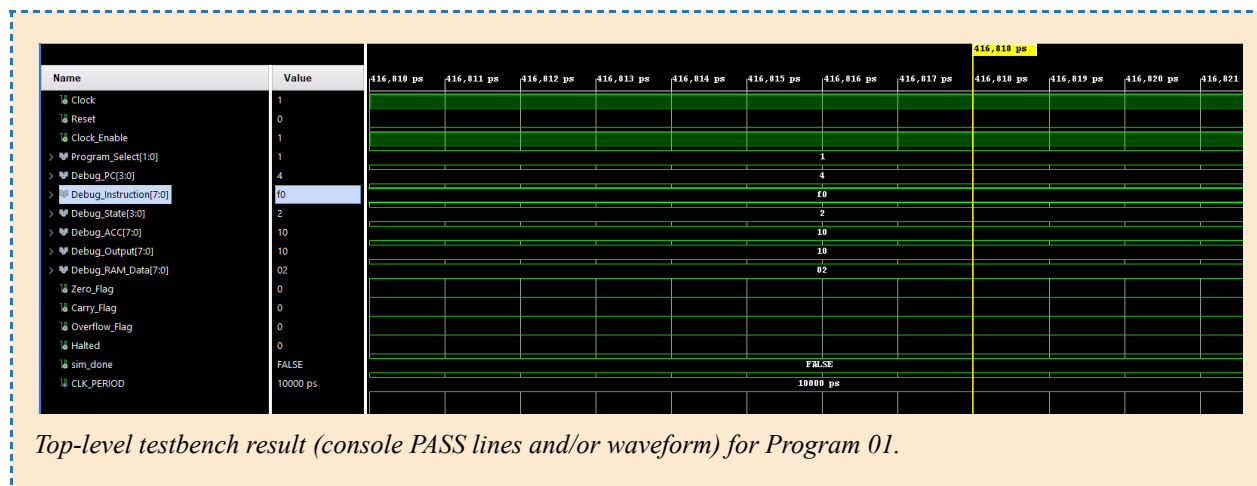


Figure 17: Top-level testbench result (console PASS lines and/or waveform).

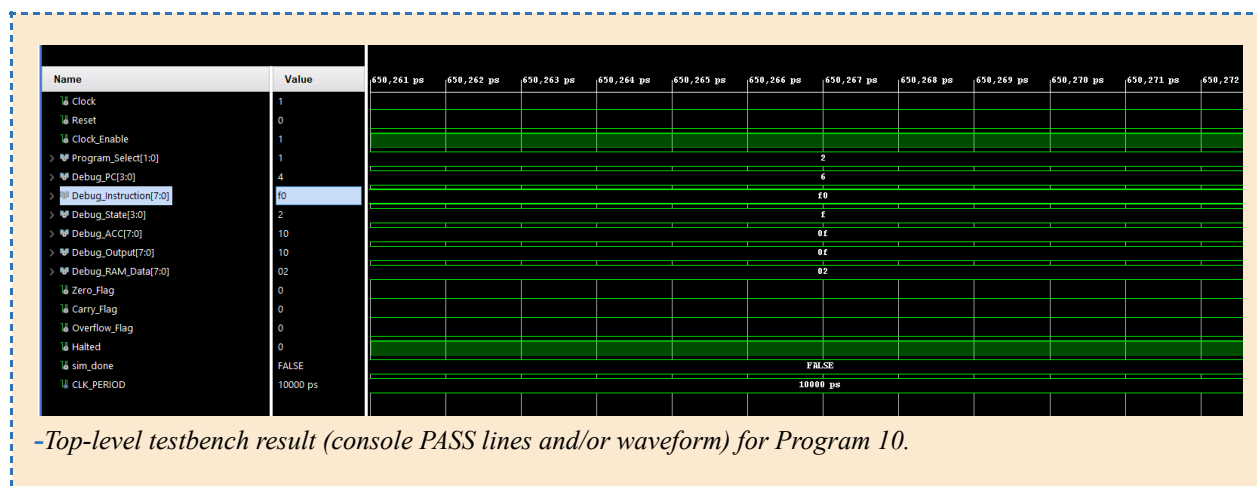


Figure 18: Top-level testbench result (console PASS lines and/or waveform).

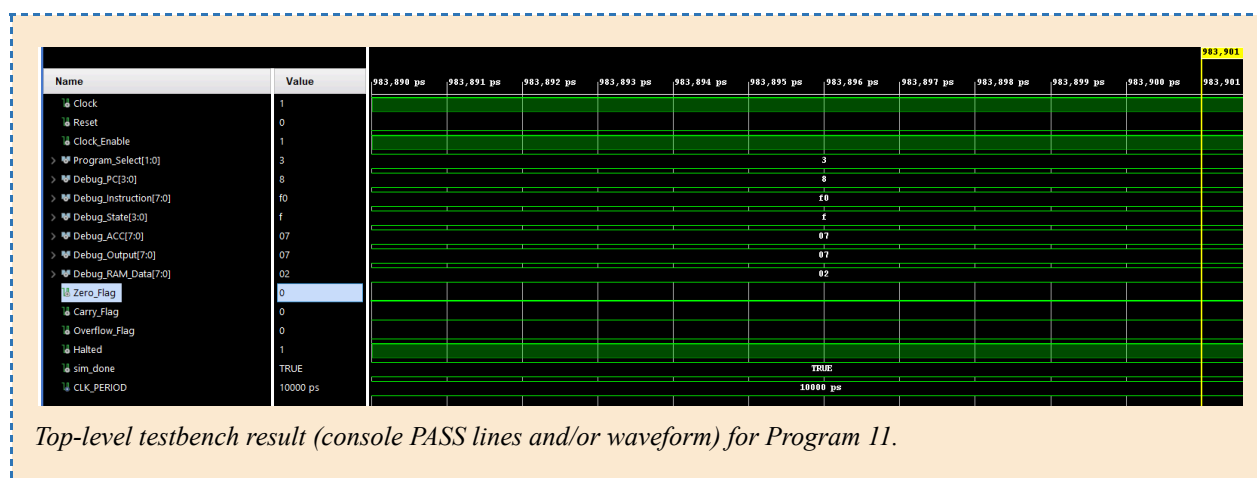


Figure 19: Top-level testbench result (console PASS lines and/or waveform).

After simulation, the design was synthesised, implemented and loaded onto the Nexys A7-100T. On the board, each program was selected with the switches and advanced using the

single-step button and free-run mode; the seven-segment display showed the program counter, state and output value, and the final output of every program matched the simulated value. Overall the CPU meets its specification: all instructions execute correctly in simulation, the results are reproduced on hardware, and the demonstration programs confirm the full fetch–decode–execute cycle including memory access and conditional branching.

Sources/References

- [1] Xilinx / AMD, Vivado Design Suite User Guide. Available: <https://docs.amd.com>
- [2] Digilent, Nexys A7 Reference Manual. Available: <https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual>
- [3] Digilent, Nexys A7 Master XDC (constraints file), GitHub. Available: <https://github.com/Digilent/digilent-xdc>
- [4] IEEE, IEEE Standard VHDL Language Reference Manual, IEEE Std 1076.
- [5] IEEE, IEEE Standard for VHDL Multivalued Logic (std_logic_1164) and numeric_std Packages, IEEE Std 1164.
- [6] [Team A8 project repository], GitHub. Available: [https://github.com/MoezMufti/SS26_HWE_Lab_Team_A8/tree/main]